

Task: Upgrade the Job Search Relevance Engine

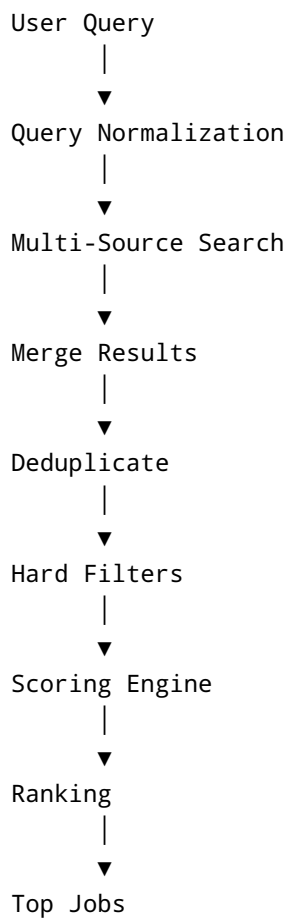
Objective

The current job aggregation pipeline returns many irrelevant jobs because external APIs perform broad keyword searches and our ranking primarily relies on resume matching. We need to redesign the search pipeline so that users only see jobs that are genuinely relevant to the role they searched for.

The goal is to maximize **precision first**, then optimize recall.

High-Level Architecture

Implement the following pipeline:



The system should reject irrelevant jobs before they ever reach the ranking stage.

Phase 1 — Query Normalization

Create a dedicated module.

Suggested location:

```
backend/  
  search/  
    query_normalizer.py
```

Responsibilities:

- Normalize job titles.
- Expand common abbreviations.
- Generate aliases.
- Remove unnecessary punctuation.
- Standardize technology names.

Examples:

```
Sr. → Senior  
  
Jr. → Junior  
  
React.js → React  
  
Node JS → Node.js  
  
Py Dev → Python Developer
```

Generate multiple search variants.

Example:

Input:

```
Python Developer
```

Generate:

```
Python Developer
Python Engineer
Backend Python Engineer
Software Engineer Python
Django Developer
FastAPI Developer
Flask Developer
```

Store mappings inside a configurable dictionary instead of hardcoding them throughout the codebase.

Phase 2 — Multi-Variant Search

Instead of sending one query to APIs:

```
Python Developer
```

Search using every generated alias.

Merge all returned jobs.

Deduplicate before scoring.

Phase 3 — Title Guardrail (Mandatory)

Implement a hard title similarity filter.

Location:

```
ranking/title_match.py
```

Use:

- RapidFuzz token_set_ratio
- token_sort_ratio
- token overlap

Avoid raw Levenshtein distance.

Requirements:

If

Title Similarity < 40%

Reject the job immediately.

Do NOT score it.

Examples:

Searching:

Python Developer

Accept:

Python Engineer

Backend Python Developer

Software Engineer (Python)

Reject:

Sales Executive

Marketing Manager

Python Faculty

Content Writer

Phase 4 — Negative Keyword Filters

Create configurable exclusion rules.

Example:

Searching software jobs should automatically reject:

Faculty
Trainer
Teacher
Tutor
Counsellor
Sales
Marketing
Recruiter
HR
Business Development
Customer Support

unless the user explicitly searched for those roles.

Keep this list configurable.

Phase 5 — Experience Matching

Extract experience from:

- title
- description

Create buckets:

Intern

0-2 Years

2-5 Years

5-8 Years

8+ Years

Examples:

Searching:

Junior Python Developer

Reject:

Principal Engineer
Engineering Director
10+ Years Required

Searching:

Senior Backend Engineer

Reject internships.

Phase 6 — Employment Type Matching

Normalize:

Full-time
Internship
Contract
Part-time
Temporary
Freelance

If user requests

Internship

Do not return full-time jobs.

Phase 7 — Remote & Location Normalization

Normalize:

```
WFH
```

```
Remote
```

```
Remote (US)
```

```
Remote (EU)
```

```
Hybrid
```

```
Onsite
```

Normalize locations.

Example:

```
Bangalore
```

```
↓
```

```
Bengaluru
```

```
↓
```

```
Karnataka
```

```
↓
```

```
India
```

Handle aliases.

Phase 8 — Deduplication

Current URL-based deduplication is insufficient.

Instead create a fingerprint using:

```
normalized_company
```

```
normalized_title
```

normalized_location

posting_week

Jobs from different APIs that represent the same opening should merge into a single listing.

Preserve multiple apply URLs.

Phase 9 — Posting Freshness

Discard jobs older than:

45 days

Implement freshness decay.

Suggested weights:

Today	= 100
1 Day	= 98
3 Days	= 95
7 Days	= 85
14 Days	= 65
30 Days	= 35
45 Days	= 10
60+ Days	= 0

Phase 10 — Company Quality

Assign a lightweight quality score.

Prioritize:

- Direct employer
- Verified company

Lower:

- Recruiters
- Unknown agencies

Do not remove recruiter jobs completely.

Only lower their ranking.

Phase 11 — Description Quality

Compute a confidence score.

Very short descriptions (<200 chars)

↓

Lower confidence.

Long complete descriptions

↓

Higher confidence.

This score should influence ranking only slightly.

Phase 12 — Skill Extraction

Extract technologies from the job description.

Examples:

Languages

Frameworks

Cloud

Databases

DevOps

ML

Compare:

Query Skills

↓

Job Skills

Resume matching should NOT be the primary relevance signal.

Phase 13 — Composite Scoring

Implement a modular scoring engine.

Suggested structure:

```
ranking/  
  
    scorer.py  
  
    title_score.py  
  
    skill_score.py  
  
    freshness.py  
  
    company_score.py  
  
    resume_score.py  
  
    semantic_score.py
```

Mandatory filters happen first.

Only surviving jobs get scored.

Suggested weights:

```
30% Title Match  
20% Query Skill Match  
15% Resume Match  
10% Semantic Similarity  
10% Freshness  
5% Company Quality  
5% Salary Match  
3% Description Quality  
2% Personalization
```

Weights should be configurable.

Phase 14 — Explainability

Each returned job should include:

```
relevance_score  
matched_skills  
title_similarity  
experience_match  
freshness  
why_matched
```

Example:

Why this job?

- ✓ Python
- ✓ FastAPI
- ✓ PostgreSQL
- ✓ Docker
- ✓ AWS
- ✓ Posted 2 days ago
- ✓ Remote
- ✓ Matches requested experience

The frontend should be able to display this information without recomputing it.

Phase 15 — API Improvements

Improve API integrations.

JSearch

Implement as primary provider.

Jooble

Implement as secondary provider.

Adzuna

Expand country support to every officially supported Adzuna market.

Findwork.dev

Add as a developer-focused source.

All providers should return a common normalized Job model.

Phase 16 — Resume Matching

Current issue:

Resume matching can rank irrelevant jobs highly.

New behavior:

Resume matching is only applied AFTER:

- title filtering
- experience filtering
- employment filtering

Resume score should never override title relevance.

Phase 17 — Future Semantic Search

Design the ranking engine so semantic embeddings can be plugged in later.

Create an interface like:

```
SemanticScorer  
  
score(query, job)
```

For now this can return 0 or use a placeholder implementation.

Do not tightly couple it to the rest of the ranking logic.

Configuration

Move thresholds into configuration.

Examples:

```
TITLE_THRESHOLD  
  
MAX_JOB_AGE
```

TITLE_WEIGHT

SKILL_WEIGHT

NEGATIVE_KEYWORDS

COMPANY_WEIGHTS

EXPERIENCE_BUCKETS

Avoid magic numbers.

Code Quality Requirements

- Keep modules small and single-purpose.
 - Add type hints.
 - Add docstrings.
 - Avoid duplicate logic.
 - Write unit tests for:
 - title similarity
 - normalization
 - deduplication
 - experience extraction
 - scoring
 - Do not break existing API contracts unless absolutely necessary.
 - Maintain backward compatibility where possible.
 - Ensure performance remains acceptable for large result sets (hundreds to thousands of jobs).
-

Success Criteria

The updated pipeline should ensure that:

- Searching "Python Developer" never returns Sales, HR, Marketing, or unrelated jobs.
- Jobs are ranked primarily by title and skill relevance, not just resume overlap.
- Duplicate listings from multiple providers are merged.
- Stale jobs are filtered out.
- Every job includes an explanation of why it was recommended.
- The ranking engine is modular, configurable, testable, and ready for future semantic search and personalization.